GenomicRanges - Basic GRanges Usage

Kasper D. Hansen

Dependencies

This document has the following dependencies:

```
library(GenomicRanges)
```

Use the following commands to install these packages in R.

```
source("http://www.bioconductor.org/biocLite.R")
biocLite(c("GenomicRanges"))
```

Corrections

Improvements and corrections to this document can be submitted on its GitHub in its repository.

Other Resources

- The vignettes from the GenomicRanges webpage.
- The package is described in a paper in PLOS Computational Biology (Lawrence et al. 2013).

DataFrame

The *S4Vectors* package introduced the `DataFrame` class. This class is very similar to the base `data.frame` class from R, but it allows columns of any class, provided a number of required methods are supported. For example, `DataFrame` can have `IRanges` as columns, unlike `data.frame`:

```
ir <- IRanges(start = 1:2, width = 3)
df1 <- DataFrame(iranges = ir)
df1
```

```
## DataFrame with 2 rows and 1 column
##     iranges
##   <IRanges>
## 1    [1, 3]
## 2    [2, 4]
```

```
df1$iranges
```

```
## IRanges of length 2
##     start end width
## [1]     1   3     3
## [2]     2   4     3
```

```
df2 <- data.frame(iranges = ir)
df2
```

```
##   iranges.start iranges.end iranges.width
## 1             1           3             3
## 2             2           4             3
```

In the `data.frame` case, the `IRanges` gives rise to 4 columns, whereas it is a single column when a `DataFrame` is used.

Think of this as an expanded and more versatile class.

GRanges, metadata

`GRanges` (unlike `IRanges`) may have associated metadata. This is immensely useful. The formal way to access and set this metadata is through `values` or `elementMetadata` or `mcols`, like

```
gr <- GRanges(seqnames = "chr1", strand = c("+", "-", "+"),
              ranges = IRanges(start = c(1,3,5), width = 3))
values(gr) <- DataFrame(score = c(0.1, 0.5, 0.3))
gr
```

```
## GRanges object with 3 ranges and 1 metadata column:
##       seqnames    ranges strand |     score
##          <Rle> <IRanges>  <Rle> | <numeric>
##   [1]     chr1    [1, 3]      + |       0.1
##   [2]     chr1    [3, 5]      - |       0.5
##   [3]     chr1    [5, 7]      + |       0.3
##   -------
##   seqinfo: 1 sequence from an unspecified genome; no seqlengths
```

A much easier way to set and access metadata is through the `$` operator

```
gr$score
```

```
## [1] 0.1 0.5 0.3
```

```
gr$score2 = gr$score * 0.2
gr
```

```
## GRanges object with 3 ranges and 2 metadata columns:
##       seqnames    ranges strand |     score    score2
##          <Rle> <IRanges>  <Rle> | <numeric> <numeric>
##   [1]     chr1    [1, 3]      + |       0.1      0.02
##   [2]     chr1    [3, 5]      - |       0.5       0.1
##   [3]     chr1    [5, 7]      + |       0.3      0.06
##   -------
##   seqinfo: 1 sequence from an unspecified genome; no seqlengths
```

findOverlaps

`findOverlaps` works exactly as for `IRanges`. But the `strand` information can be confusing. Let us make an example

```
gr2 <- GRanges(seqnames = c("chr1", "chr2", "chr1"), strand = "*",
               ranges = IRanges(start = c(1, 3, 5), width = 3))
gr2
```

```
## GRanges object with 3 ranges and 0 metadata columns:
##       seqnames    ranges strand
##          <Rle> <IRanges>  <Rle>
##   [1]     chr1    [1, 3]      *
##   [2]     chr2    [3, 5]      *
##   [3]     chr1    [5, 7]      *
##   -------
##   seqinfo: 2 sequences from an unspecified genome; no seqlengths
```

```
gr
```

```
## GRanges object with 3 ranges and 2 metadata columns:
##       seqnames    ranges strand |     score    score2
##          <Rle> <IRanges>  <Rle> | <numeric> <numeric>
##   [1]     chr1    [1, 3]      + |       0.1      0.02
##   [2]     chr1    [3, 5]      - |       0.5       0.1
##   [3]     chr1    [5, 7]      + |       0.3      0.06
##   -------
##   seqinfo: 1 sequence from an unspecified genome; no seqlengths
```

Note how the `ranges` in the two `GRanges` object are the same coordinates, they just have different `seqnames` and `strand`. Let us try to do a standard `findOverlaps`:

```
findOverlaps(gr, gr2)
```

```
## Hits object with 4 hits and 0 metadata columns:
##       queryHits subjectHits
##       <integer>   <integer>
##   [1]         1           1
##   [2]         2           1
##   [3]         2           3
##   [4]         3           3
##   -------
##   queryLength: 3
##   subjectLength: 3
```

Notice how the `*` strand overlaps both `+` and `-`. There is an argument `ignore.strand` to `findOverlaps` which will … ignore the strand information (so `+` overlaps `-`). Several other functions in `GenomicRanges` have an `ignore.strand` argument as well.

subsetByOverlaps

A common operation is to select only certain ranges from a `GRanges` which overlap something else. Enter the convenience function `subsetByOverlaps`

```
subsetByOverlaps(gr, gr2)
```

```
## GRanges object with 3 ranges and 2 metadata columns:
##       seqnames    ranges strand |     score    score2
##          <Rle> <IRanges>  <Rle> | <numeric> <numeric>
##   [1]     chr1    [1, 3]      + |       0.1      0.02
##   [2]     chr1    [3, 5]      - |       0.5       0.1
##   [3]     chr1    [5, 7]      + |       0.3      0.06
##   -------
##   seqinfo: 1 sequence from an unspecified genome; no seqlengths
```

makeGRangesFromDataFrame

A common situation is that you have data which looks like a `GRanges` but is really stored as a classic `data.frame`, with `chr`, `start` etc. The `makeGRangesFromDataFrame` converts this `data.frame` into a `GRanges`. An argument tells you whether you want to keep any additional columns.

```
df <- data.frame(chr = "chr1", start = 1:3, end = 4:6, score = 7:9)
makeGRangesFromDataFrame(df)
```

```
## GRanges object with 3 ranges and 0 metadata columns:
##       seqnames    ranges strand
##          <Rle> <IRanges>  <Rle>
##   [1]     chr1    [1, 4]      *
##   [2]     chr1    [2, 5]      *
##   [3]     chr1    [3, 6]      *
##   -------
##   seqinfo: 1 sequence from an unspecified genome; no seqlengths
```

```
makeGRangesFromDataFrame(df, keep.extra.columns = TRUE)
```

```
## GRanges object with 3 ranges and 1 metadata column:
##       seqnames    ranges strand |     score
##          <Rle> <IRanges>  <Rle> | <integer>
##   [1]     chr1    [1, 4]      * |         7
##   [2]     chr1    [2, 5]      * |         8
##   [3]     chr1    [3, 6]      * |         9
##   -------
##   seqinfo: 1 sequence from an unspecified genome; no seqlengths
```

Biology usecases

Here are some simple usecases with pseudo-code showing how we can accomplish various tasks with `GRanges` objects and functionality.

Biology usecase I

Suppose we want to identify transcription factor (TF) binding sites that overlaps known SNPs.

Input objects are
snps: a `GRanges` (of width 1)
TF: a `GRanges`

pseudocode:

```
findOverlaps(snps, TF)
```

(watch out for strand)

Biology usecase II

Suppose we have a set of differentially methylation regions (DMRs) (think genomic regions) and a set of CpG Islands and we want to find all DMRs within 10kb of a CpG Island.

Input objects are
dmrs: a `GRanges`
islands: a `GRanges`

pseudocode:

```
big_islands <- resize(islands, width = 20000 + width(islands), fix = "ce
findOverlaps(dmrs, big_islands)
```

(watch out for strand)

SessionInfo

```
## R version 3.2.1 (2015-06-18)
## Platform: x86_64-apple-darwin13.4.0 (64-bit)
## Running under: OS X 10.10.5 (Yosemite)
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## attached base packages:
## [1] stats4    parallel  methods   stats     graphics  grDevices utils
## [8] datasets  base
##
## other attached packages:
## [1] GenomicRanges_1.20.5 GenomeInfoDb_1.4.2   IRanges_2.2.7
## [4] S4Vectors_0.6.3      BiocGenerics_0.14.0  BiocStyle_1.6.0
## [7] rmarkdown_0.7
##
## loaded via a namespace (and not attached):
##  [1] digest_0.6.8    formatR_1.2     magrittr_1.5    evaluate_0.7.2
##  [5] stringi_0.5-5   XVector_0.8.0   tools_3.2.1     stringr_1.0.0
##  [9] yaml_2.1.13     htmltools_0.2.6 knitr_1.11
```

References

Lawrence, Michael, Wolfgang Huber, Hervé Pagès, Patrick Aboyoun, Marc Carlson, Robert Gentleman, Martin T Morgan, and Vincent J Carey. 2013. “Software for computing and annotating genomic ranges.” *PLoS Computational Biology* 9 (8): e1003118. doi:10.1371/journal.pcbi.1003118.